TRANSACCIONES

DAW 1 – Bases de Datos

Práctica 3.6

Modificación de bases de datos con confirmación de transacciones

Marco Valiente Rodríguez
Mvalienter2501@educantabria.es

Índice

Contenido

Índice 1

Planteamiento del Problema 2

a. Muestra el estado de la variable @@autocommit y asegúrate de que está a 1. Explica con tus palabras lo que implica esto. 2

b. Explica con tus palabras la utilidad de las sentencias “COMMIT” y “ROLLBACK”. 2

c. Propón un ejemplo práctico en el que puede resultar de utilidad emplear una política de bloqueo de registros. Explica con tus palabras lo que implica dicha política. 3

d. Transacciones y Consultas 4

1. Transacción con Rollback 4
2. Transacción con COMMIT 7
3. Consulta 1 9
4. Consulta 2 10
5. Consulta 3 11
6. Consulta 4 12
7. Consulta 5 13

Bibliografía 14

Planteamiento del Problema

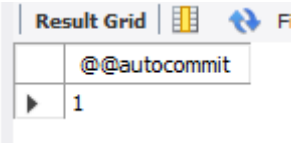
En esta práctica comenzamos con las transacciones en MySQL, que consisten en las confirmaciones que se realizan como método de seguridad antes de realizar una transacción. Así mismo, si una transacción es confirmada, no hay vuelta atrás para restaurar los cambios, por lo que consultar que se ha cambiado antes de darle a confirmar posee su importancia.

a. Muestra el estado de la variable @@autocommit y asegúrate de que está a 1. Explica con tus palabras lo que implica esto.

Para ver el valor de esta variable, podemos obtener dicho valor con un simple comando usando SELECT.

```
SELECT @@autocommit;
```

Y tras su ejecución, podemos ver el valor de nuestra variable.



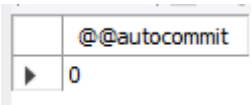
	@@autocommit
▶	1

Que @@autocommit esté en 1 significa que los cambios realizados no se pueden deshacer usando ROLLBACK. Esta variable tiene un valor 1 si está activado o un valor 0 si se encuentra desactivado.

Para nuestro ejercicio, vamos a desactivar previamente esta variable convirtiéndola en 0.

```
SET @@autocommit = 0;
```

Tras este comando, tenemos que efectivamente, la variable @@autocommit ahora es 0.



	@@autocommit
▶	0

b. Explica con tus palabras la utilidad de las sentencias "COMMIT" y "ROLLBACK".

La sentencia COMMIT es la confirmación de los cambios realizados, mientras que ROLLBACK es lo opuesto, la cancelación de los cambios realizados a la base de datos. La potestad de poder dictar si se desean guardar o desechar cambios tienen más de una ventaja sobre su contraparte de autoguardado, aunque en una primera instancia parezca simplemente algo redundante y simplista.

Poder determinar cuándo se guardan los cambios y cuando se desechan es útil sobre todo cuando se están realizando cambios en la base de datos con una gran cantidad de información, ya que en estas situaciones si ocurre algún error, se puede dar marcha atrás, y sólo se confirma si está todo correcto.

Sin embargo, esto está lejos de ser su única ventaja, ya que la capacidad de decidir cuándo se hace un COMMIT también mejora la velocidad de la realización de cambios de información dentro de la base de datos o previene que no se consuman demasiados recursos, que, aunque en MySQL no tarda demasiado, si importa a grandes escalas. Esto se debe a que cada vez que se realiza un cambio, se actualiza inmediatamente la base de datos, lo que significa que, si hay varios cambios seguidos, la base de datos se tiene que actualizar varias veces, en vez de solo una vez, si por ejemplo simplemente hacemos un COMMIT at final.

Y esto, así mismo, tiene otra desventaja, ya que va en contra del modelo denominado ACID, que significa por sus siglas en inglés, atomicy-consistency-isolation-durability. Este modelo se centra en que las transacciones sean cambios indivisibles creados en línea de tal manera que si, por ejemplo, una base de datos de un banco realiza una retirada de dinero mientras que en otro lado se hace un ingreso, que estas dos no se sobrescriban una a la otra, un problema que sí se tendría si a la hora de realizar una transacción se necesita de usar un COMMIT.

Es por esto que modelos como MySQL tienen por defecto encendido que se use el autocommit por defecto. Pero para nosotros, al ser nosotros mismos aquellos que controlan la base de datos y los que acceden a la misma, no es algo que nos deba preocupar.

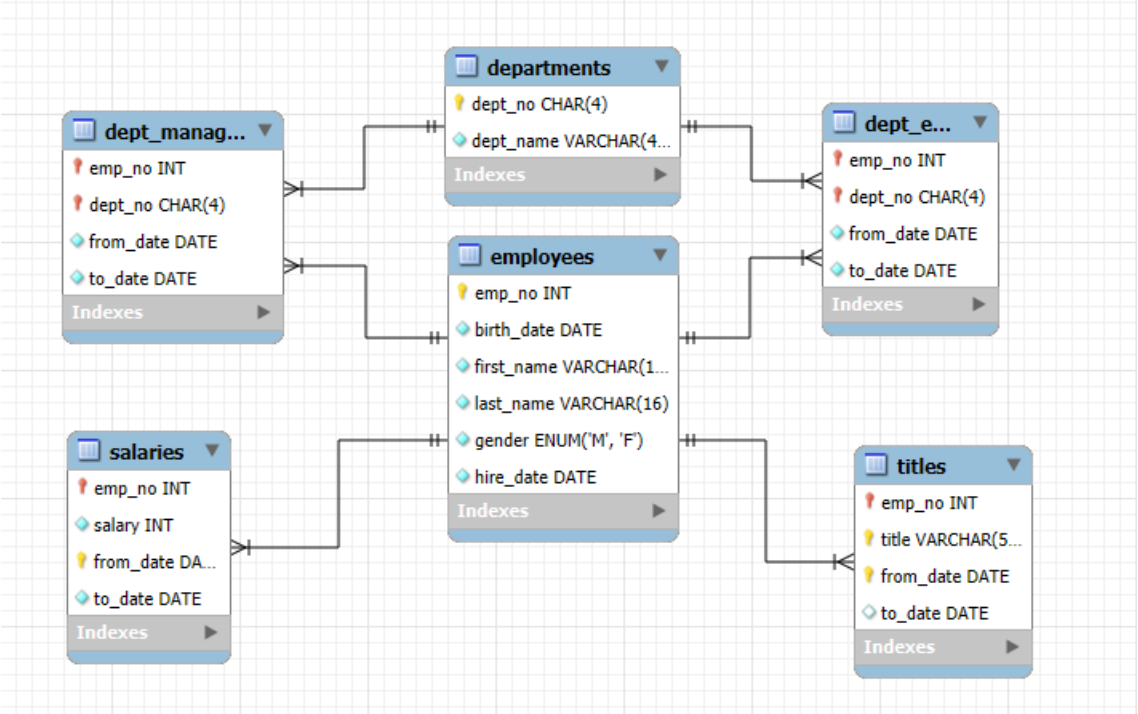
c. Propón un ejemplo práctico en el que puede resultar de utilidad emplear una política de bloqueo de registros. Explica con tus palabras lo que implica dicha política.

Un ejemplo de una base de datos donde sea deseable poner estas medidas, por lo comentado anteriormente, es una base de datos que posea datos estáticos y que no sean editados con mucha frecuencia, sino que además sean imprescindibles para mantener lo más alejado posible de errores.

Como un posible ejemplo de estas características, podemos describir una base de datos de pacientes de un hospital con sus historiales médicos, los cuales son esenciales mantener protegidos, ya que podría ser un gravísimo desastre si estos datos fuesen editados de forma involuntaria o se perdiesen, ya que se volverían casi irrecuperables, además de que no se deben realizar cambios dentro de los historiales previos con frecuencia, ya que estos cambios fueron realizados en el pasado, de ahí su nombre historial.

d. Transacciones y Consultas

Antes de continuar con la práctica, recordamos cuales son nuestras tablas en esta base de datos. Más adelante en el ejercicio, vamos a hacer consultas de esta base de datos, por lo que es conveniente recordar dicha estructura.



1. Transacción con Rollback

Inicia una transacción y aumenta el salario en 1.000€ a todas las mujeres contratadas el primer año de funcionamiento de la empresa y que actualmente trabajen en ella. Muestra en una consulta el salario que tenían antes de la transacción, el salario que tienen tras realizar el UPDATE y por último descarta la transacción. Muestra de nuevo los salarios y razona si el resultado se ajusta a lo previsto.

Comenzamos aquí con las transacciones que se nos piden hacer, en este caso, con el aumento del salario de las mujeres contratadas el primer año de la empresa que sigan trabajando en esta. Como nuestro primer desafío, tenemos que determinar exactamente cual es el primer año de funcionamiento de la empresa, y esto lo podemos intentar determinar a través de la hire_date más antingua de la que se disponga.

```
SELECT e.hire_date FROM employees e ORDER BY e.hire_date ASC LIMIT 1;
```

Ejecutando este comando, podemos obtener la siguiente fecha.

	hire_date
▶	1985-02-18

Pero, aun así, vamos a insertar esta selección como método de ordenación dentro de nuestra propia transacción. Aunque, antes de realizar dicha transacción, vamos a comprobar exactamente cuánto es el salario que tenían antes.

```
SELECT e.first_name, e.last_name, e.hire_date, s.salary
FROM employees e
JOIN salaries s ON e.emp_no = s.emp_no
WHERE s.to_date > CURRENT_DATE()
AND e.gender = 'F'
AND TIMESTAMPDIFF(YEAR, (SELECT e2.hire_date FROM employees e2 ORDER BY
e2.hire_date ASC LIMIT 1), e.hire_date) < 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

first_name	last_name	hire_date	salary
Bezalel	Simmel	1985-11-21	72527
Sumant	Peac	1985-02-18	94409
Sanjiv	Zschoche	1986-02-04	78478
Erez	Ritzmann	1985-07-09	62921

Y ahora realizamos la actualización de los datos, incrementando estos salarios en 1000€. Para realizar este cambio, se requieren de múltiples cambios. El primero de todos es el de actualizar la tabla anterior para guardar el histórico de los cambios de salario.

Antes de nada, debemos empezar a realizar los cambios.

```
START TRANSACTION;
```

Con este comando, podemos luego hacer ROLLBACK cuando queramos cancelar los cambios.

```
UPDATE salaries s
JOIN employees e ON e.emp_no = s.emp_no
SET to_date = CURRENT_DATE()
WHERE s.to_date > CURRENT_DATE()
AND e.gender = 'F'
AND TIMESTAMPDIFF(YEAR, (SELECT e2.hire_date FROM employees e2 ORDER BY
e2.hire_date ASC LIMIT 1), e.hire_date) < 1;
```

Y podemos luego de esto leer cómo se han guardado los cambios originales con una búsqueda cambiando a que seleccionemos la fecha actual.

```
SELECT s.emp_no, s.salary, s.from_date, s.to_date
FROM employees e
JOIN salaries s ON e.emp_no = s.emp_no
WHERE s.to_date = CURRENT_DATE()
AND e.gender = 'F'
AND TIMESTAMPDIFF(YEAR, (SELECT e2.hire_date FROM employees e2 ORDER BY
e2.hire_date ASC LIMIT 1), e.hire_date) < 1;
```

Y con esto conseguimos los datos que acabamos de cambiar.

Result Grid | Filter Rows: | Export: | Wrap

emp_no	salary	from_date	to_date
10002	72527	2001-08-02	2026-02-25
10009	94409	2002-02-14	2026-02-25
10053	78478	2001-11-11	2026-02-25
10076	62921	2002-07-14	2026-02-25

Efectivamente, los datos han sido cambiados de forma correcta.

Y con esta selección, se puede combinar para la creación de un INSERT que cree nuestro nuevo registro.

```
INSERT INTO `salaries`
SELECT s.emp_no,
       s.salary + 1000,
       CURRENT_DATE(),
       '9999-01-01'
FROM salaries s
JOIN employees e ON e.emp_no = s.emp_no
WHERE s.to_date = CURRENT_DATE()
AND e.gender = 'F'
AND TIMESTAMPDIFF(YEAR, (SELECT e2.hire_date FROM employees e2 ORDER BY
e2.hire_date ASC LIMIT 1), e.hire_date) < 1;
```

Y tras su ejecución, podemos ver el resultado final ejecutando la selección original de los salarios actuales de esos empleados.

Result Grid | Filter Rows:

first_name	last_name	hire_date	salary
Bezalel	Simmel	1985-11-21	73527
Sumant	Peac	1985-02-18	95409
Sanjiv	Zschoche	1986-02-04	79478
Erez	Ritzmann	1985-07-09	63921

Efectivamente, los nuevos datos están introducidos de forma correcta.

Por último, no deseamos realizar estos cambios, así que hacemos ROLLBACK.

```
ROLLBACK;
```

Result Grid | Filter Rows:

first_name	last_name	hire_date	salary
Bezalel	Simmel	1985-11-21	72527
Sumant	Peac	1985-02-18	94409
Sanjiv	Zschoche	1986-02-04	78478
Erez	Ritzmann	1985-07-09	62921

Y como comprobamos, nos ha salido de forma correcta la reversión de los cambios que han sido mostrados antes de este ejercicio.

2. Transacción con COMMIT

Inicia una transacción y aumenta el salario un 5% a todos los empleados varones cuyo título contenga la palabra "Senior" y actualmente trabajen en la empresa. Muestra en una consulta el salario que tenían antes de la transacción, el salario que tienen tras realizar el UPDATE y por último confirma la transacción. Muestra de nuevo los salarios y razona si el resultado se ajusta a lo previsto.

Para la realización de esta tarea, tenemos que realizar una búsqueda similar a la que usamos en el apartado anterior.

```
SELECT e.first_name, e.last_name, t.title, s.salary
FROM employees e
JOIN salaries s ON e.emp_no = s.emp_no
JOIN titles t ON t.emp_no = e.emp_no
WHERE s.to_date > CURRENT_DATE()
AND t.to_date > CURRENT_DATE()
AND e.gender = 'M'
AND t.title like '%Senior%';
```

Con esta búsqueda se obtienen todos los que vamos a querer seleccionar.

first_name	last_name	title	salary
Georgi	Facello	Senior Engineer	88958
Parto	Bamford	Senior Engineer	43311
Chirstian	Koblick	Senior Engineer	74057
Kyoichi	Maliniak	Senior Staff	94692
Patricio	Bridgland	Senior Engineer	54423
Eberhardt	Terkki	Senior Staff	68901
Yongqiao	Berztiss	Senior Engineer	66313
Otmar	Herbst	Senior Engineer	77777
Elvis	Demeyer	Senior Engineer	88806
Karsten	Joslin	Senior Engineer	56689
Alain	Chappelet	Senior Engineer	68755

Y a partir de esto, realizamos nuestra transacción.

```
START TRANSACTION;
```

Y realizamos el UPDATE e INSERT que vamos a querer realizar.

```
UPDATE salaries s
JOIN employees e ON e.emp_no = s.emp_no
JOIN titles t ON t.emp_no = e.emp_no
SET s.to_date = CURRENT_DATE()
WHERE s.to_date > CURRENT_DATE()
AND t.to_date > CURRENT_DATE()
AND e.gender = 'M'
AND t.title LIKE '%Senior%';
```

Y con estos cambios realizados, creamos la nueva tabla. No hace falta mostrar el registro anterior, ya que hemos comprobado anteriormente que funciona.

```
INSERT INTO `salaries`
SELECT s.emp_no,
       s.salary + s.salary*0.05,
       CURRENT_DATE(),
       '9999-01-01'
FROM salaries s
JOIN employees e ON e.emp_no = s.emp_no
JOIN titles t ON t.emp_no = e.emp_no
WHERE s.to_date = CURRENT_DATE()
AND t.to_date > CURRENT_DATE()
AND e.gender = 'M'
AND t.title LIKE '%Senior%';
```

Y ahora comprobamos con la consulta anterior, que todos los cambios han sido realizados correctamente.

Result Grid | Filter Rows: | Export: | Wrap Ce

first_name	last_name	title	salary
Georgi	Facello	Senior Engineer	93406
Parto	Bamford	Senior Engineer	45477
Chirstian	Koblick	Senior Engineer	77760
Kyoichi	Maliniak	Senior Staff	99427
Patricio	Bridgland	Senior Engineer	57144
Eberhardt	Terkki	Senior Staff	72346
Yongqiao	Berztiss	Senior Engineer	69629
Otmar	Herbst	Senior Engineer	81666
Elvis	Demeyer	Senior Engineer	93246
Karsten	Joslin	Senior Engineer	59523
Alain	Chappelet	Senior Engineer	72193

Result 26

Los cambios han sido realizados de forma correcta, efectivamente estos cambios son un incremento de un 5%.

Y con esto finalizado, realizamos el comando que es característico de MySQL cuando se tiene que la variable global @@autocommit es puesta a 0.

```
COMMIT;
```

7 10:01:15 COMMIT 0 row(s) affected

Y está completado. Nuestra transacción ahora debería fallar si se intenta dar un ROLLBACK.

first_name	last_name	title	salary
Georgi	Facello	Senior Engineer	93406
Parto	Bamford	Senior Engineer	45477
Chirstian	Koblick	Senior Engineer	77760
Kyoichi	Maliniak	Senior Staff	99427
Patricio	Bridgland	Senior Engineer	57144
Eberhardt	Terkki	Senior Staff	72346
Yongqiao	Berztiss	Senior Engineer	69629
Otmar	Herbst	Senior Engineer	81666
Elvis	Demeyer	Senior Engineer	93246
Karsten	Joslin	Senior Engineer	59523
Alain	Chappelet	Senior Engineer	72193

Result 3

Output

Action Output

#	Time	Action
7	10:01:15	COMMIT
8	10:03:45	ROLLBACK
9	10:03:51	SELECT e.first_name, e.last_name, t.title, s.salary FROM employees

Y podemos comprobar que dicha hipótesis se prueba como cierta. Los cambios han sido realizados dentro de las tablas de la base de datos y ya no hay marcha atrás.

3. Consulta 1

Muestra el nombre, apellido, fecha de contratación, salario, título y departamento de los 5 empleados más recientes.

Comenzamos ahora con las consultas de la base de datos, que pueden tener los cambios que se nos han requerido en el ejercicio anterior.

Usamos el siguiente código:

```
SELECT e.first_name, e.last_name, e.hire_date, s.salary, t.title,
d.dept_name
FROM employees e
JOIN dept_emp de ON de.emp_no = e.emp_no
JOIN departments d ON de.dept_no = d.dept_no
JOIN salaries s ON s.emp_no = e.emp_no
JOIN titles t ON t.emp_no = e.emp_no
WHERE s.to_date > CURRENT_DATE()
AND t.to_date > CURRENT_DATE()
AND de.to_date > CURRENT_DATE()
ORDER BY e.hire_date DESC
LIMIT 5;
```

Y con su ejecución, podemos ver a los 5 empleados más nuevos.

first_name	last_name	hire_date	salary	title	dept_name
Lillian	Haddadi	1999-04-30	50032	Staff	Research
Suzette	Pettey	1997-05-19	96646	Assistant Engineer	Production
Sailaja	Desikan	1996-11-05	82715	Staff	Sales
Tuval	Kalloufi	1995-12-15	98302	Senior Engineer	Production
Shahaf	Famili	1995-08-22	41348	Engineer	Development

Tras esta búsqueda, podemos ver al Senior Tuval, que es uno de los empleados que han recibido un incremento de su sueldo por nuestras transacciones anteriores.

4. Consulta 2

Lista el nombre, apellido y salario actual de los empleados cuyo salario actual es superior a la media de todos los salarios actuales.

```
SET @salarioMedioActual = CAST((SELECT AVG(s2.salary) FROM salaries s2
WHERE s2.to_date > CURRENT_DATE()) AS DECIMAL(7,2));
SELECT e.first_name, e.last_name, s.salary, @salarioMedioActual AS
'Salario Medio Actual'
FROM employees e
JOIN salaries s ON s.emp_no = e.emp_no
WHERE s.salary > @salarioMedioActual
AND s.to_date > CURRENT_DATE();
```

Usando una subselección de todos los salarios medios actuales y tomando ventaja de la función AVG(), podemos obtener una tabla con nuestros resultados deseados.

Como método de visualización, tenemos a la derecha de cada tabla, la constante que es el Salario Medio. Hay varias formas que se podría haber expresado este valor, y la mejor forma para realizarlo, es asignando este valor a una variable, para que, a la hora de mostrarlo, no se necesite hacer la búsqueda más de una vez. Sin variables, esta búsqueda sería de O(n^2), ya que la subconsultas se harían dentro de las mismas para una información que podríamos mantener simplemente con una variable. Aquí tenemos los datos correctamente consultados:

first_name	last_name	salary	Salario Medio Actual
Georgi	Facello	93406	74841.37
Chirstian	Koblick	77760	74841.37
Kyoichi	Maliniak	99427	74841.37
Tzvetan	Zielinski	88070	74841.37
Sumant	Peac	94409	74841.37
Duangkaew	Piveteau	80324	74841.37
Kazuhito	Cappelletti	77935	74841.37
Cristinel	Bouloucos	99651	74841.37
Kazuhide	Peha	84672	74841.37

5. Consulta 3

Encuentra al empleado que menos tiempo ha tardado en pasar de "Assistant Engineer" a "Senior Engineer", mostrando el número exacto de días que tardó.

Para esta consulta, tenemos más potestad de poder elegir qué se va a mostrar de nuestro preguntado empleado, y para eso, vamos a mostrar tanto su nombre, como sus títulos, y su número exacto de días también.

```
SELECT e.first_name, e.last_name, t1.title, t2.title, t1.from_date, t1.to_date, t2.from_date, t2.to_date, TIMESTAMPDIFF(DAY, t1.from_date, t1.to_date) AS 'Days it took'
FROM employees e
JOIN titles t1 ON e.emp_no = t1.emp_no
JOIN titles t2 ON e.emp_no = t2.emp_no
WHERE t1.title = 'Assistant Engineer'
AND t2.title = 'Senior Engineer'
AND t2.from_date > t1.from_date
ORDER BY TIMESTAMPDIFF(DAY, t1.from_date, t1.to_date) ASC
LIMIT 1;
```

Necesitamos hacer una comprobación que no es muy intuitiva para la realización correcta de este ejercicio, que es que la fecha del titulo de Senior Engineer viene después de la de Assistant Engineer.

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

first_name	last_name	title	title	from_date	to_date	from_date	to_date	TIMESTAMPDIFF(DAY, t1.from_date, t1.to_date)
Sumant	Peac	Assistant Engineer	Senior Engineer	1985-02-18	1990-02-18	1995-02-18	9999-01-01	1826

Y este es nuestro empleado deseado.

6. Consulta 4

Para cada departamento, muestra el nombre, apellido y fecha de contratación del empleado más antiguo que sigue trabajando allí.

Para realizar esta consulta, se requiere de la realización de otra subconsulta cuyo propósito sea que nuestro from_date sea el seleccionado.

```
SELECT d.dept_name, e.first_name, e.last_name, e.hire_date
FROM employees e
JOIN dept_emp de ON de.emp_no = e.emp_no
JOIN departments d ON de.dept_no = d.dept_no
WHERE de.to_date > CURRENT_DATE()
AND (de.from_date) IN (SELECT MIN(de2.from_date) FROM dept_emp de2
    WHERE de2.dept_no = d.dept_no AND de2.to_date > CURRENT_DATE());
```

Tras su realización, conseguimos el siguiente resultado.

Result Grid | Filter Rows: | Export: | Wrap C

dept_name	first_name	last_name	hire_date
Customer Service	Huan	Lortz	1989-09-20
Development	Kwee	Schusler	1986-02-26
Finance	Alejandro	McAlpine	1991-06-26
Human Resources	Eberhardt	Terkki	1985-10-20
Marketing	Berhard	McFarlin	1987-04-13
Production	Zhongwei	Rosen	1986-10-30
Quality Management	Sumant	Peac	1985-02-18
Research	Tzvetan	Zielinski	1989-02-10
Sales	Charlene	Brattka	1987-08-07

Y efectivamente los datos concuerdan.

7. Consulta 5

Cuenta cuántos empleados tienen actualmente el título "Engineer" en cada departamento.

Por último, tenemos que hacer una agrupación para resolver este ejercicio.

```
SELECT d.dept_name, COUNT(e.emp_no) AS 'Personas con título Engineer'
FROM employees e
JOIN dept_emp de ON e.emp_no = de.emp_no
JOIN departments d ON de.dept_no = d.dept_no
JOIN titles t ON e.emp_no = t.emp_no
WHERE t.title = 'Engineer'
AND t.to_date > CURRENT_DATE()
AND de.to_date > CURRENT_DATE()
GROUP BY d.dept_name
ORDER BY COUNT(e.emp_no) DESC;
```

Y a partir de esto, obtenemos nuestros datos. Cabe destacar que no estamos buscando que contengan Engineer en su nombre.

Result Grid | Filter Rows: | Export:

dept_name	Personas con título Engineer
Development	6
Production	3
Quality Management	1

Bibliografía

- Recurso del curso de Desarrollo de Aplicaciones Web, Documento del Curso 2025/26.
- https://mariadb.com/docs/connectors/mariadb-connector-c/api-functions/mysql_autocommit